In [16]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

In [17]:
```
df = pd.read_csv('SteelPlatesFaults.csv')
df.shape
```

Out[17]: (1941, 34)

In [18]:
```
df.info()
```

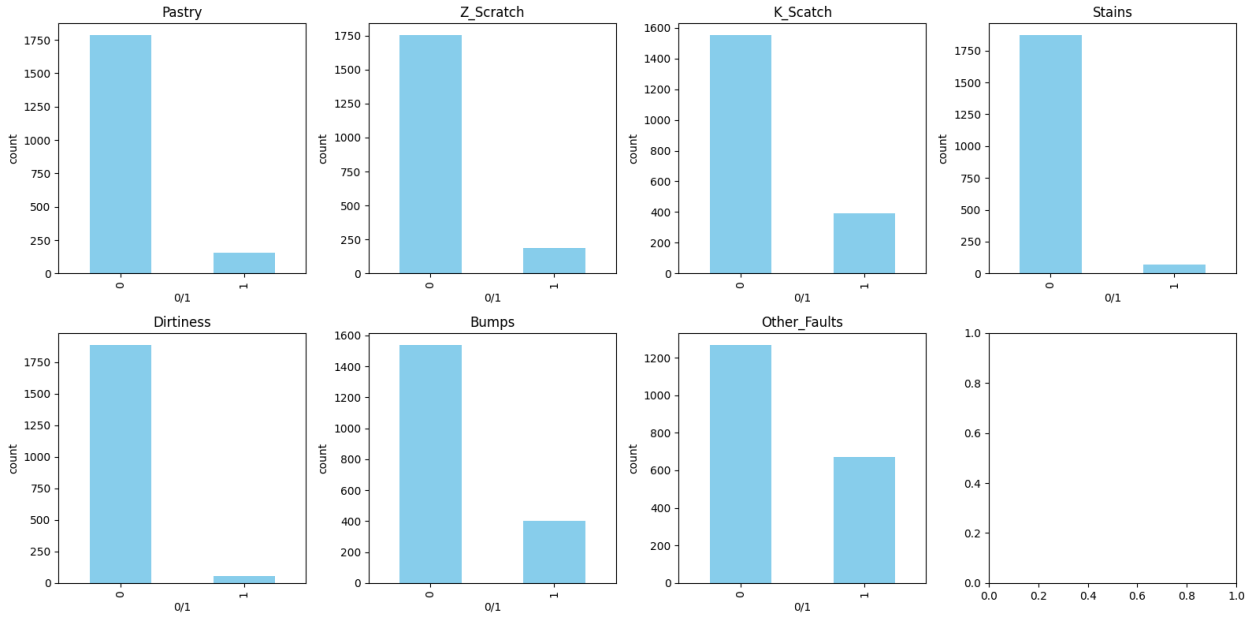
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1941 entries, 0 to 1940
Data columns (total 34 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   X_Minimum               1941 non-null   int64  
 1   X_Maximum               1941 non-null   int64  
 2   Y_Minimum               1941 non-null   int64  
 3   Y_Maximum               1941 non-null   int64  
 4   Pixels_Areas            1941 non-null   int64  
 5   X_Perimeter             1941 non-null   int64  
 6   Y_Perimeter             1941 non-null   int64  
 7   Sum_of_Luminosity       1941 non-null   int64  
 8   Minimum_of_Luminosity   1941 non-null   int64  
 9   Maximum_of_Luminosity   1941 non-null   int64  
 10  Length_of_Conveyer      1941 non-null   int64  
 11  TypeOfSteel_A300        1941 non-null   int64  
 12  TypeOfSteel_A400        1941 non-null   int64  
 13  Steel_Plate_Thickness   1941 non-null   int64  
 14  Edges_Index             1941 non-null   float64
 15  Empty_Index             1941 non-null   float64
 16  Square_Index            1941 non-null   float64
 17  Outside_X_Index         1941 non-null   float64
 18  Edges_X_Index           1941 non-null   float64
 19  Edges_Y_Index           1941 non-null   float64
 20  Outside_Global_Index    1941 non-null   float64
 21  LogOfAreas              1941 non-null   float64
 22  Log_X_Index             1941 non-null   float64
 23  Log_Y_Index             1941 non-null   float64
 24  Orientation_Index       1941 non-null   float64
 25  Luminosity_Index        1941 non-null   float64
 26  SigmoidOfAreas          1941 non-null   float64
 27  Pastry                  1941 non-null   int64  
 28  Z_Scratch               1941 non-null   int64  
 29  K_Scatch                1941 non-null   int64  
 30  Stains                  1941 non-null   int64  
 31  Dirtiness               1941 non-null   int64  
 32  Bumps                   1941 non-null   int64  
 33  Other_Faults            1941 non-null   int64  
dtypes: float64(13), int64(21)
memory usage: 515.7 KB
```

In [ ]:
```
df.head()
```

In [ ]:
```
df.describe().round(2)
```

In [ ]:
```
df.isnull().sum()
```

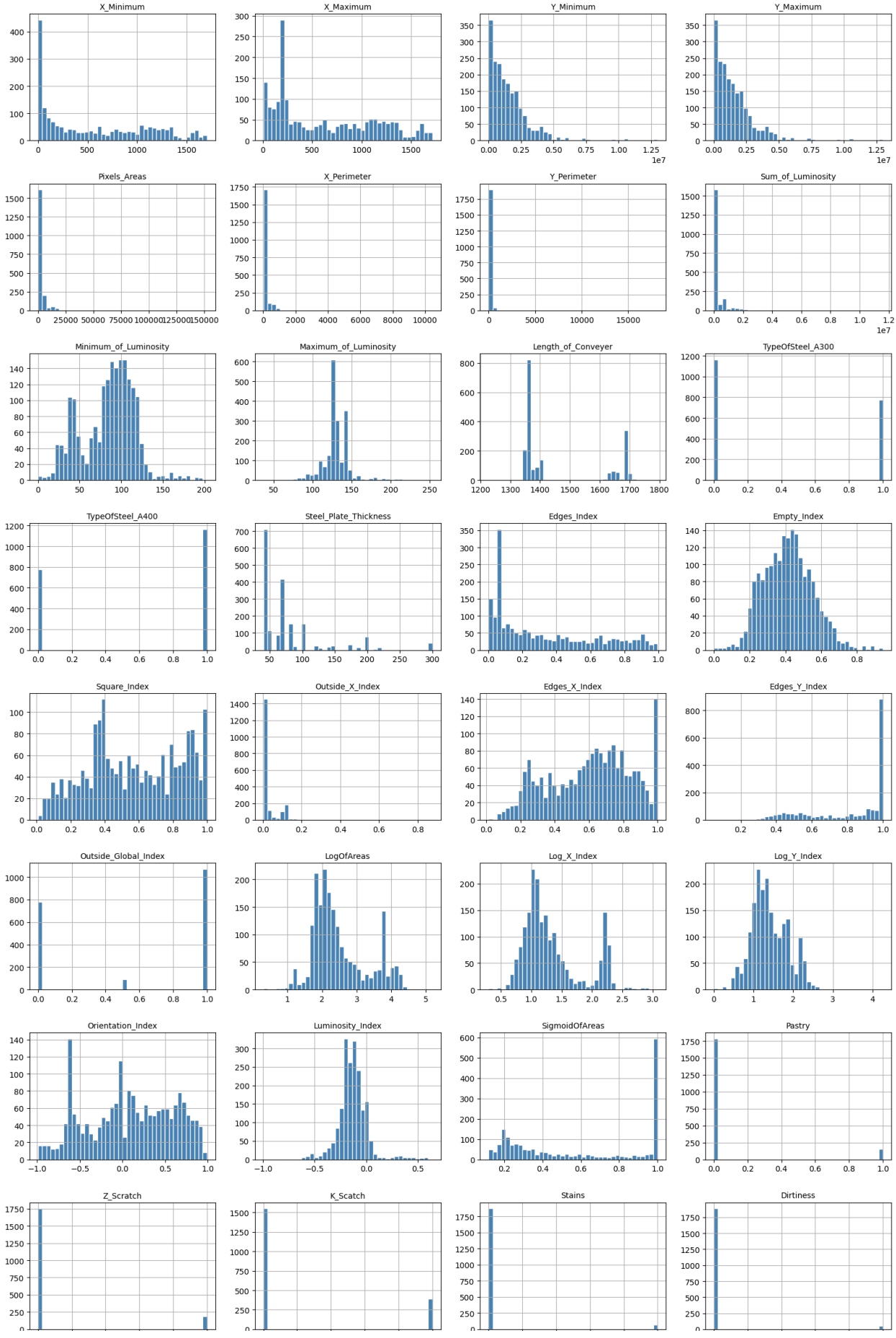
In [22]:
```
target_cols = ['Pastry', 'Z_Scratch', 'K_Scatch', 'Stains', 'Dirtiness', 'Bump

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()
for i, col in enumerate(target_cols):
    df[col].value_counts().plot.bar(ax=axes[i], color='skyblue')
    axes[i].set_title(col)
    axes[i].set_xlabel('0/1')
    axes[i].set_ylabel('count')
plt.tight_layout()
plt.show()
```




Виден дисбаланс классов, целых тарелок больше поврежденных

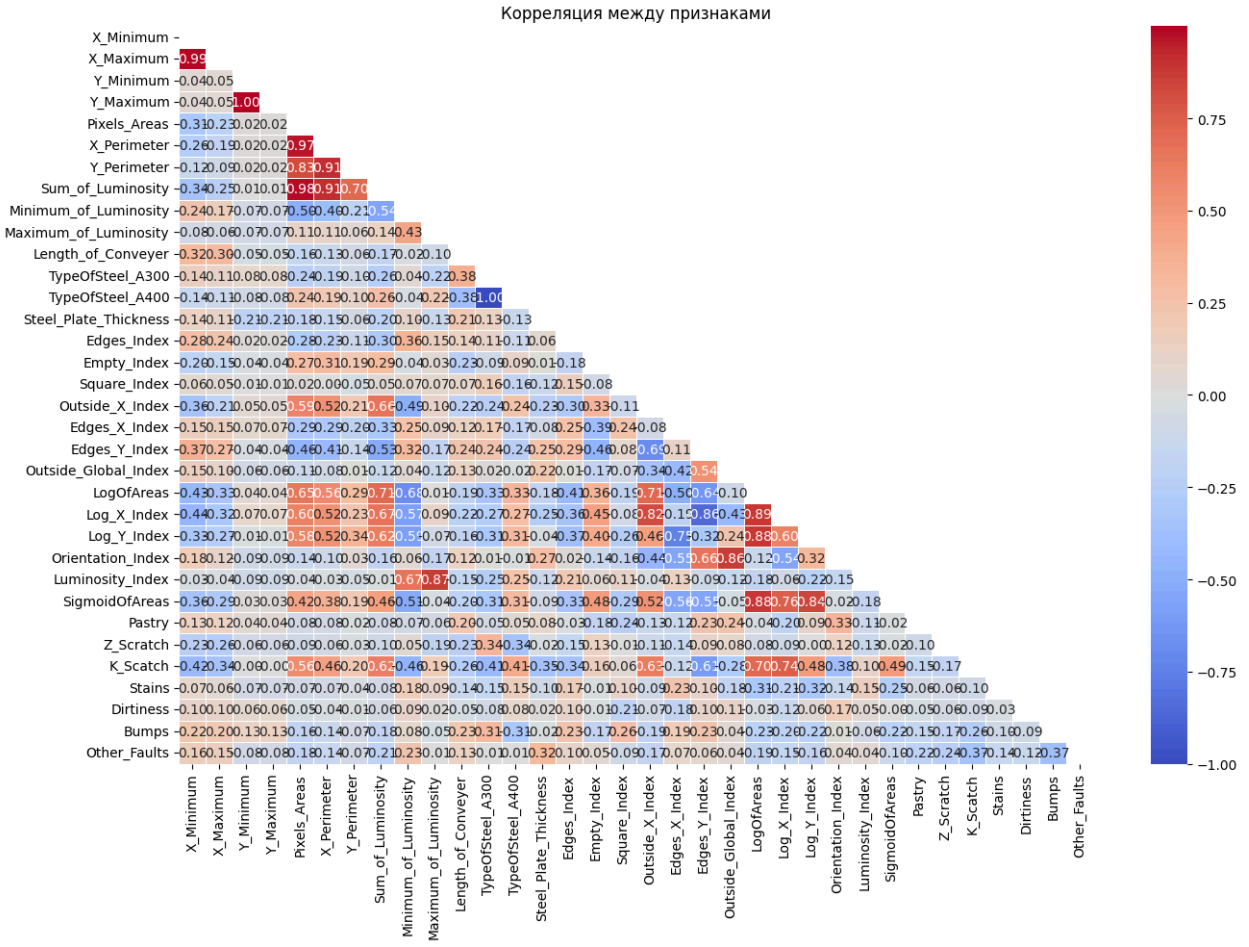
In [23]:
```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()

n_cols = len(numeric_cols)
n_rows = (n_cols + 3) // 4
fig, axes = plt.subplots(n_rows, 4, figsize=(16, n_rows * 3))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    df[col].hist(bins=40, ax=axes[i], color='steelblue', edgecolor='white')
    axes[i].set_title(col, fontsize=10)

for j in range(len(numeric_cols), len(axes)):
    axes[j].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```


X_Minimum
X_Maximum
Y_Minimum
Y_Maximum
1e7
1e7
Pixels_Areas
X_Perimeter
Y_Perimeter
Sum_of_Luminosity
1e7
Minimum_of_Luminosity
Maximum_of_Luminosity
Length_of_Conveyer
TypeOfSteel_A300
TypeOfSteel_A400
Steel_Plate_Thickness
Edges_Index
Empty_Index
Square_Index
Outside_X_Index
Edges_X_Index
Edges_Y_Index
Outside_Global_Index
LogOfAreas
Log_X_Index
Log_Y_Index
Orientation_Index
Luminosity_Index
SigmoidOfAreas
Pastry
Z_Scratch
K_Scatch
Stains
Dirtiness

У Pixels_Areas, X_Perimeter, Y_Perimeter, Sum_of_Luminosity есть выбросы, удалим их

In [24]:
```
plt.figure(figsize=(14, 10))
corr = df[numeric_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm', center=0,
plt.title('Корреляция между признаками')
plt.tight_layout()
plt.show()
```




In [25]:
```
from sklearn.model_selection import train_test_split

target_cols = ['Pastry', 'Z_Scratch', 'K_Scatch', 'Stains', 'Dirtiness', 'Bump
X = df.drop(columns=target_cols)
y = df[target_cols]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [26]:
```
cols_to_drop = [
    'Log_X_Index', 'Log_Y_Index',
    'X_Maximum', 'Y_Maximum',
]
```

```
X_train_reduced = X_train.drop(columns=cols_to_drop)
X_test_reduced = X_test.drop(columns=cols_to_drop)
```

In [27]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [28]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import f1_score, hamming_loss

lr = MultiOutputClassifier(LogisticRegression(max_iter=1000, random_state=42))
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)


print("Логистическая регрессия")
print(f"macro F1: {f1_score(y_test, y_pred_lr, average='macro'):.4f}")
print(f"micro F1: {f1_score(y_test, y_pred_lr, average='micro'):.4f}")
print(f"Hamming Loss: {hamming_loss(y_test, y_pred_lr):.4f}")

print("\nСлучайный лес")
print(f"macro F1: {f1_score(y_test, y_pred_rf, average='macro'):.4f}")
print(f"micro F1: {f1_score(y_test, y_pred_rf, average='micro'):.4f}")
print(f"Hamming Loss: {hamming_loss(y_test, y_pred_rf):.4f}")
```

```
Логистическая регрессия
macro F1: 0.6667
micro F1: 0.6946
Hamming Loss: 0.0775

Случайный лес
macro F1: 0.7485
micro F1: 0.7583
Hamming Loss: 0.0613
```

Гипотеза 1 подтвердилась. У RandomForest macro F1 = 0.7485,а у LogReg 0.6667. Случайный лес лучше улавливает нелинейные зависимости

In [32]:
```
from sklearn.model_selection import cross_val_score

lr_cv = MultiOutputClassifier(LogisticRegression(max_iter=1000, random_state=4
scores_lr = cross_val_score(lr_cv, X_train_scaled, y_train, cv=5, scoring='f1_
print(f"LogReg CV macro F1: {scores_lr.mean():.4f}")

rf_cv = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
scores_rf = cross_val_score(rf_cv, X_train, y_train, cv=5, scoring='f1_macro')
print(f"RF CV macro F1: {scores_rf.mean():.4f} ")
```

```
LogReg CV macro F1: 0.6642
RF CV macro F1: 0.7474
```

In [34]:
```
from sklearn.model_selection import GridSearchCV

param_grid_lr = {
    'estimator__C': [0.01, 0.1, 1, 10],
    'estimator__penalty': ['l1', 'l2'],
    'estimator__solver': ['liblinear']
}

grid_lr = GridSearchCV(
    MultiOutputClassifier(LogisticRegression(max_iter=1000)),
    param_grid_lr,
    cv=5,
    scoring='f1_macro'
)
grid_lr.fit(X_train_scaled, y_train)

print("Лучшие параметры LogReg:", grid_lr.best_params_)
print("Лучший CV f1_macro:", grid_lr.best_score_)


param_grid_rf = {
    'max_depth': [5, 10, 15, 20, None],
    'n_estimators': [50, 100, 200]
}

grid_rf = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid_rf,
    cv=5,
    scoring='f1_macro'
)
grid_rf.fit(X_train, y_train)

print("Лучшие параметры RF:", grid_rf.best_params_)
print("Лучший CV f1_macro:", grid_rf.best_score_)
```

```
Лучшие параметры LogReg: {'estimator__C': 10, 'estimator__penalty': 'l1', 'esti
mator__solver': 'liblinear'}
Лучший CV f1_macro: 0.6669306665015079
Лучшие параметры RF: {'max_depth': 20, 'n_estimators': 100}
Лучший CV f1_macro: 0.7550448702435967
```

Гипотеза 5 подтвердилась. GridSearch выбрал L1 регуляризациюкак самую лучшую

Гипотеза 6 не подтвердилась. Наилучшей оказалась глубина 20